

Université de Versailles Saint-Quentin-en-Yvelines

Licence 2 Informatique

Année universitaire 2025–2026

Projet de morphing d'images

Étudiants :

Benayoune Abdelmoumen

Aggoun Dylane

11 janvier 2026

Table des matières

1 Lire et afficher des images contenues dans un fichier

1.1 Introduction

Nous avons commencé par créer le répertoire du projet ainsi qu'un fichier texte permettant de documenter notre travail. Ce fichier avait pour objectif d'être réécrit proprement en $\text{\LaTeX}$ par la suite. La méthode *Diviser pour régner* étant déjà appliquée dans l'énoncé du projet, nous avons suivi les différentes étapes imposées.

1.2 Création de l'arborescence

Les dossiers destinés à contenir les images ont été créés conformément aux consignes du projet, afin d'assurer une organisation claire et structurée des fichiers.

1.3 Makefile et outils externes

Un **Makefile** a été écrit afin de permettre l'utilisation de l'outil **ImageMagick**. Celui-ci a été installé sur le système, puis testé à l'aide des commandes définies dans le **Makefile**.

1.4 Conversion des images

Un fichier `convertir.c` a été créé. Le programme est écrit en langage C et utilise la fonction **system** afin d'appeler directement les commandes **ImageMagick**. Le **Makefile** a ensuite été modifié pour permettre la compilation et l'exécution automatique de ce programme.

1.5 Structures de données

N'ayant pas encore une vision claire des structures nécessaires pour la triangulation, nous avons retenu des structures simples :

- une structure pour stocker une image (largeur, hauteur, range et pixels composés de trois composantes) ;
- une liste chaînée pour stocker les points de base, contenant les coordonnées (x, y) du point courant et un pointeur vers le point suivant.

1.6 Programme principal et affichage

Étant donné qu'il était demandé de réaliser un programme pour la conversion mais également d'écrire des fonctions spécifiques, nous avons créé un fichier `morphing.c` servant de programme principal. Ce fichier contient des fonctions d'allocation et de désallocation de mémoire ainsi que des fonctions de stockage d'images. Les bibliothèques **uvsqgraphics** ont été intégrées comme demandé.

Une fonction d'affichage a été ajoutée afin de montrer les deux images côte à côte, séparées par un espace de 100 pixels.

2 Sélectionner les couples de points de base entre les deux images

Pour cette étape, nous avons appliqué la méthode *Diviser pour régner* en définissant nos propres règles de fonctionnement :

- organisation de la fenêtre en plusieurs zones : image de départ en haut à gauche, image d'arrivée en haut à droite, affichage des coordonnées au centre et boutons en bas ;
- sélection des points par un clic sur l'image de gauche puis sur celle de droite (aucune action n'est effectuée si l'ordre n'est pas respecté) ;
- affichage de points numérotés lors de chaque sélection ;

- bouton **Sauver** permettant d'écrire les coordonnées dans un fichier `points.txt` ;
- bouton **Quitter** pour fermer le programme ;
- suppression d'un couple de points si l'utilisateur clique sur un point déjà existant.

3 Calcul et génération des images intermédiaires

3.1 Lecture des points

Une fois les couples de points sauvegardés dans le fichier `points.txt`, nous avons écrit une fonction permettant de relire ce fichier et de stocker les couples de points dans une structure de données adaptée.

3.2 Structures utilisées

Nous avons défini :

- une structure **Couple** contenant un point dans l'image de départ et un point dans l'image d'arrivée ;
- une structure **Triangle** contenant trois indices de points.

3.3 Calcul des points intermédiaires

Les points intermédiaires sont calculés pour un paramètre α donné à l'aide de la formule suivante :

$$P_{\text{inter}} = (1 - \alpha)P_{\text{départ}} + \alpha P_{\text{arrivée}}$$

3.4 Triangulation et calcul des pixels

Afin de déformer correctement l'image, une triangulation des points intermédiaires est mise en place. Le rectangle de l'image est d'abord découpé en deux triangles, puis les autres points sont ajoutés progressivement.

Plusieurs fonctions mathématiques ont été implémentées :

- calcul du déterminant ;
- test d'appartenance d'un point à un triangle ;
- calcul des coordonnées barycentriques.

Pour chaque pixel de l'image intermédiaire, nous déterminons le triangle correspondant, calculons ses coordonnées barycentriques, puis projetons ce pixel dans les triangles des images de départ et d'arrivée. Les couleurs sont ensuite interpolées avec le même paramètre α .

3.5 Génération des images

Le processus est répété pour k variant de 0 à N , avec $\alpha = k/N$. On obtient ainsi $N + 1$ images intermédiaires sauvegardées au format PPM dans le dossier `Images_Intermediaires` sous la forme `img_000.ppm`, `img_001.ppm`, etc.

4 Génération du film final

Le programme final s'exécute sous la forme :

```
morphing image_depart.ppm image_arrivee.ppm N
```

Après la sélection des points, le programme génère automatiquement les images intermédiaires. La conversion en vidéo est effectuée à l'aide d'un appel système à l'outil `ffmpeg` :

```
ffmpeg -framerate 3 -i Images_Intermediaires/img_%03d.ppm -c:v libx264 -pix_fmt  
yuv420p morphing.mp4
```

Cette commande permet de produire une vidéo d'environ dix secondes montrant la transformation progressive de l'image de départ vers l'image d'arrivée. L'ensemble du processus est entièrement automatisé par une seule commande.